
Binary Reverse Engineering Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *October 31st, 2019*

Introduction

1. Why reverse?

- (a) Source code is often unavailable: firmware, games, malware, proprietary protocol implementations, vendor drivers.
- (b) To understand execution at the machine level, which respects no abstraction: the CPU executes what the compiler emitted, not what the developer intended.
- (c) To verify source claims when auditing: compilers transform code, and the shipped binary is the ground truth (optimizations, inlining, dead code elimination).
- (d) To recover structure: symbol stripping, obfuscation, and packing commonly accompany anything valuable to protect.

2. When would you reverse (give example situations)?

- (a) Fuzzing an undocumented file format: understanding the parser directs the fuzzer and seeds the corpus (see the fuzzing lecture).
- (b) Vulnerability research on closed source targets (browsers, drivers, IoT firmware) where a large part of the attack surface is compiled code.
- (c) Building a sanitizer/instrumentation harness: you need to know where the input crosses the trust boundary before you can hook it.
- (d) Writing an exploit: kernel structures, driver interaction, and mitigation layout are read from the binary and from the running system.
- (e) Malware analysis: capability identification, C2 protocol recovery, IOC production.
- (f) Interoperability: reimplementing a protocol when the vendor refuses to document it.

3. How does reversing fit into the larger VR picture?

- (a) It is the static-analysis half of vulnerability research when no source exists; it fills the same role source auditing does for open targets (compare the Comparison of VR Techniques in the source auditing lecture).
- (b) It feeds dynamic analysis: reversing produces breakpoints, input grammars, harness entry points, and expected invariants for debugging and fuzzing.
- (c) It is iterative with exploitation: superseding assumptions (allocator layout, mitigation status) requires reading the shipped code.

4. What are the legal aspects of reversing?

- (a) This is not legal advice. Laws differ by jurisdiction (US DMCA, EU Software Directive, etc) and are applied unevenly.
- (b) Interoperability and research exceptions generally exist (e.g. reverse engineering to achieve interoperability in the EU; fair-use argumentation in the US, cf. *Sega v. Accolade*, *Sony v. Connectix*), but anti-circumvention provisions and EULAs complicate them.
- (c) Practical rules: reverse what you own or are licensed to run, do not redistribute their code or derived assets, do not attack third parties from your findings, and get written authorization for anything that touches systems you do not own.

Overview of How Compilers Work

1. Go from C to binary and back. Give them an idea of what happens under the hood.
 - (a) A C source file is preprocessed (macro expansion, header inclusion, conditional compilation), then lowered through successively less abstract IRs (in LLVM: Clang AST → LLVM IR → selection DAG → MachineInstr → MCInst) and encoded into object code.
 - (b) The reverse path, taken by decompilers, loses information at every lowering step. Names, types, comments, control-flow intent are gone. This is why decompilation is lossy and why guided reversing is needed.
 - (c) Anything the compiler knows from the source (struct layout, constant expressions) may be folded into immediates; recovery ones are inferred by the decompiler as best-effort.
2. Front end of compiler, (Lexical analysis (lex), grammar(yacc), semantic analysis(type checking))
 - (a) Lexical analysis: characters to tokens.
 - (b) Syntax analysis: tokens to a parse tree per a grammar.
 - (c) Semantic analysis: type checking, scope resolution, implicit conversions.
 - (d) Front end output (AST) is lowered to IR; the IR platform-independence is what makes retargetable code generation possible.
3. Back end of compiler, (optimization, register spilling, code generation)
 - (a) Optimization at IR level: dead code elimination, constant propagation, inlining, loop unrolling, vectorization.
 - (b) Instruction selection: IR to machine instructions; register allocation: instructions to physical registers with spilling to stack slots when registers run out.
 - (c) Code generation and emission: scheduling, peephole, object file writing (relocations, symbol tables).

- (d) Implication for reversing: optimization changes code shape (inlined functions disappear, spilled values are untracked). When possible compare -O0 through the shipped optimization level to classify structures.
- (e) Idioms that identify an optimization at a glance: a `jmp` over an unreachable block (dead code), a call whose result is stored but never read (eliminated), a loop replaced by a closed-form expression (strength reduction, loop unrolling), and a `memset/memcpy` call where the source had a scalar loop. Recognizing these is most of reading optimized output.
- (f) `-fomit-frame-pointer` removes the frame-pointer chain that makes a stack walk trivial; the unwind tables (`.eh_frame`, `.pdata`) are then the only reliable call-graph ground truth. Keep the tables even when the code is stripped.
- (g) Debug info is a separate, optional artifact: DWARF on Linux/macOS (`-g`, `.debug_*` sections), PDB on Windows. `objcopy --only-keep-debug` splits it from the shipped binary, and the split is itself a reversing shortcut when it leaks.
- (h) Intermediate representation is readable: `clang -S -emit-llvm` prints LLVM IR, which is far closer to the source than assembly. When a vendor ships IR or bitcode (firmware, JIT caches, Swift modules), read that before the machine code.

Object Formats and Linking

1. Before code is data: object formats. ELF is the Linux/Unix standard, Mach-O the macOS/iOS one, PE the Windows one.
2. Sections: executable code (`.text/__TEXT,__text`), initialized data (`.data/__DATA,__data`), uninitialized data (`.bss`), read-only data (`.rodata/__TEXT,__cstring`), unwind/debug info.
3. Symbol tables map names to addresses; stripping removes them but the code remains. Relocations are patch points applied at link/load time; resolving them is how position-independent calls are walked.
4. Import tables (PLT/GOT on ELF, stubs and lazy binding on Mach-O) show which library functions the code calls without reversing a byte of it: usually the first stop when triaging a binary.
5. File formats are themselves parsers: malformed headers and section tables crash tools (compare fuzzing lecture: disassemblers and loaders are attack surface too).

ELF, the Linux/macOS-unix default

1. Read the header, then the program headers, then the section headers: `e_ident` identifies the class (32/64) and endianness, `e_type` distinguishes executable from shared object from core, and the program headers are what the loader actually consumes.
2. The dynamic section is the linking contract: `DT_NEEDED` lists the libraries, `DT_SYMTAB/DT_STRTAB/D` locate the symbol and hash tables, `DT_INIT/DT_INIT_ARRAY` name the constructors that run before `main`, and `DT_FLAGS` records NOW binding.

3. `PT_GNU_RELRO` marks the range the loader re-protects read-only after relocation; `PT_GNU_STACK` encodes the executable-stack request, and `PT_LOAD` entries carry the actual permissions. These are the on-disk mirror of the source auditing lecture's mitigation map.
4. The PLT/GOT pair is the import mechanism: each unresolved call goes through a stub that pushes a relocation index and jumps to the resolver; after the first call the slot holds the resolved address (lazy binding). `LD_BIND_NOW` or `-z now` forces eager resolution, and full RELRO re-protects `.got` afterwards — the RELRO row of the mitigation map.
5. `STT_GNU_IFUNC` symbols (glibc's `memcpy`, `strlen` selection) are resolved at load time by running code, so a resolver is an execution path that exists before `main`. Rare, but a real reversing and auditing surprise.
6. `PT_TLS` and the thread-pointer register (`fs/gs` on x86, `tpidr_el0` on ARM64) locate thread-local storage; a wrong `tpidr` recovery is a common decompiler failure on Android.
7. Stripping removes `.symtab`, not the dynamic symbol table: `.dynsym` still names every imported and exported function, so imports survive stripping and are the first thing to read. `--only-keep-debug` plus `--strip-debug` produces the common shipped split (debug file kept by the vendor).
8. Unwind data (`.eh_frame`, `.eh_frame_hdr`, `.ARM.exidx`) is present even in stripped binaries that were built with exceptions; it is a function-boundary oracle the stripper usually forgets to remove.

Mach-O, the macOS/iOS default

1. Load commands (`LC_SEGMENT_64`, `LC_MAIN`, `LC_DYLD_INFO`) replace ELF's program headers; segments are named `__TEXT`, `__DATA`, `__LINKEDIT`.
2. Imports go through `__stubs/` `__stub_helper` (the PLT/GOT analogue); modern binaries replace them with chained fixups (`LC_DYLD_CHAINED_FIXUPS`), where each pointer carries its own target encoding.
3. Objective-C and Swift metadata (`__objc_classlist`, `__objc_methname`, `__swift5_*`) is high-level structure the compiler emitted into the binary: method names, class layouts, and protocol conformance survive stripping, which makes a stripped Objective-C binary far easier to read than a stripped C one.
4. On iOS the shared cache (`dyld_shared_cache`) holds the system libraries pre-linked; reversing it requires the `dyld_shared_cache_util` extraction step.

PE, the Windows default

1. The DOS header's `e_lfanew` points at the NT headers; the optional header holds the entry point (`AddressOfEntryPoint`), image base, section alignment, and the data directories.
2. Imports are described by `IMAGE_IMPORT_DESCRIPTOR` entries naming DLLs and their `IMAGE_THUNK_DATA` arrays, which are the IAT slots the loader patches. Delayed and ordinal imports are the variants to recognize.

3. Exports (IMAGE_EXPORT_DIRECTORY) name the DLL's own functions: on a shipped Windows binary, exports are the most reliable symbol source.
4. RUNTIME_FUNCTION entries (.pdata) are the x64 exception/unwind table; they are machine-readable function boundaries and therefore a reversing aid even without symbols.

Tools

1. Reversing tools like Radare, IDA, Binja, objdump, etc
 - (a) Disassembly + decompilation: IDA Pro (Hex-Rays, the industry standard; paid), Binary Ninja (newer, strong ILs for scripted analysis), Ghidra (NSA open source, includes decompiler), radare2/rizin (free, scriptable, steep learning curve), Hopper (macOS).
 - (b) Command line for quick looks: objdump (binary sizes, sections, symbols, simple disasm), llvm-objdump, otool/nm/lipo on macOS, readelf on ELF.
 - (c) Dynamic analysis: gdb (gdb-multiarch recommended for remote targets), LLDB, debuggers built into the above; Frida for scriptable hooking of live processes; ltrace/strace/turbostat for syscall-level views.
 - (d) Scriptability is a core requirement: real reversing is automated (batch findings across thousands of functions; write plugins or IDAPython/gdb scripts).
 - (e) Library and framework choices: LIEF and pefile for parsing formats in scripts, Capstone/Zydis for disassembly, Keystone for assembling, Unicorn/Qiling for emulation, and angr/miasm/Triton for symbolic work. These are the primitives a custom reversing tool is built from.
 - (f) Symbol servers and signature databases (Microsoft's public symbol server, FLIRT signatures, Lumina, FunctionID) turn a stripped binary back into a named one. Reading pdb paths out of a PE or .gnu_debuglink out of an ELF is the first move of a triage.
 - (g) Debugger integration is what separates a disassembler from a reversing tool: the debugger is the ground truth from the debugging module, and a decompiler that cannot be checked against a run is a hypothesis generator.

Anti-Reversing Techniques

1. Expect the target to resist. Families to recognize and defeat:
 - (a) **Packing**: UPX, VMProtect/Themida style virtualizers, custom crypters. Detection: section entropy high in exactly one section, few imports, self-modifying memory at runtime. Defeat: run and dump (user-mode unpackers, or break at the original entry point in a debugger), or emulate.
 - (b) **Anti-debugging**: IsDebuggerPresent, PEB/proc status checks, ptrace (PT_DENY_ATTACH) on macOS, timing checks, exception filters as control flow. Defeat: patch out the calls once located, or attach before they run.

- (c) **Obfuscation:** control-flow flattening (dispatcher + state variable shape in the decompiler), opaque predicates, instruction substitution, string encryption. Defeat: patience + scripting; flattening complicates control flow, but the data flow remains traceable.
 - (d) **Integrity checks:** self-hash of code sections at runtime. Defeat: patch + recompute/resign, or break the checker call site.
- 2. Cost logic mirrors the mitigation map in the source auditing lecture: anti-reversing raises the attacker's analysis cost per-function, not the bug count. When a target is heavily obfuscated, put scripting/automation effort in first — manual reading rarely amortizes.
- 3. Concrete detection tells for the families above:
 - (a) **Packing:** section entropy near 8.0 in exactly one section, a section name that is not standard (UPX0/UPX1), few or no imports, an entry point outside the first section, a writable-and-executable section at load, and a tiny on-disk-to-in-memory size ratio.
 - (b) **Anti-debugging:** imports of `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess` with `ProcessDebugPort/ProcessDebugFlags/ProcessDebugObject`, `NtSetInformationThread` with `ThreadHideFromDebugger`, `OutputDebugString` with a last-error check, `ptrace(PT_DENY_ATTACH)` on macOS, a `rdtsc/clock_gettime` pair, and `int 2d/int 3` or `popf` used as a check.
 - (c) **Obfuscation:** control-flow flattening (one dispatcher block with a state variable compared in every successor), MBA expressions, opaque predicates that are always true but not provably so, encrypted strings decrypted at use, and instruction substitution. LLVM-based obfuscators (OLLVM, tigress) leave recognizable shapes.
 - (d) **Integrity checks:** a hash or CRC over the code section, computed and compared at startup; the comparison site is the patch point, and the recompute is the resign step.
- 4. Defeat toolkit, in cost order: patch the check once located; hide from the check (*ScyllaHide*, *TitanHide*); unpack to the original entry point (OEP) and dump; emulate (*Unicorn*/*Qiling*) so the check never runs; and symbolically simplify (*Triton*/*miasm*) when the obfuscation is arithmetic. Every one of these is itself a tooling project, which is the cost the protection is buying.
- 5. The honest framing for the report: anti-reversing is not a bug and not a mitigation. It is a tax on the *auditor*, and the correct response is a time/effort estimate, exactly as the mitigation map prices a bypass.

Architectures x86, x86_64, ARM, ARM64, MIPS, PPC

- 1. Things about arch's.
 - (a) Instruction sets come in orthogonal and non-orthogonal flavors, fixed and variable size encodings, load-store vs. memory-register models. Table:

Arch	Encoding	Notes
x86	variable (1–15 bytes)	dense, memory-register ops, hard to decode
x86_64	variable	+8 GPRs, RIP-relative addressing
ARM	fixed 4 bytes (A32)	condition codes on every instruction
Thumb2	2/4 byte mixture	ARMv7+; mode ambiguity in disassembly
ARM64	fixed 4 bytes	31 GPRs, no condition flags carry-over
MIPS	fixed 4 bytes	load-store, delay slots, canonical in embedded
PPC	fixed 4 bytes	load-store, big endian in older systems

- (b) Calling conventions: how arguments are placed, where the return value is returned, what registers are callee-saved, how structs/classes are constructed, all differ per ABI.
- System V AMD64 (Linux/macOS): args in rdi, rsi, rdx, rcx, r8, r9; return in rax; rbx, rbp, r12–r15 callee-saved; return address and frame pointer on the stack.
 - AArch64: args in x0–x7; return in x0; x19–x28 and x29 (FP) and x30 (LR) callee-saved; LR in a register (implicit push of x30 in prologue).
 - ARM32: r0–r3 args; LR in r14; condition meaningful for Thumb/ARM mode selection (LSB of function pointers indicating Thumb).
 - Knowledge of the convention where the target was compiled is a prerequisite to reading prologues, argument registers, and call sites.
- (c) Features that affect reversing: endianness (bi-endian ARM/MIPS/PPC), byte order of embedded filesystem dumps (big endian MIPS/PPC still common in embedded), SIMD/vector extensions (NEON, SSE/AVX, AltiVec) changing data-flow shape, thus decompiler capability.
- (d) ARM64 hardening shapes are visible in the prologue and epilogue and change what a stack overflow buys: `paciasp/autiasp` (pointer authentication on the saved x29/x30), `bti c` (branch-target identification, the ARM64 CFI landing pad), and `irg/stg` (memory tagging). A reversing session that does not name these will mis-read the prologue.
- (e) x86 hardening shapes: `endbr64` at every indirect-call target (CET, the counterpart of `bti c`), shadow-stack instructions, and `movaps` alignment requirements that make a pivoted stack fault before the chain runs.
- (f) ABI details that recur: x86-64 SysV passes a variadic call's vector-register count in `al`; a struct return uses a hidden pointer in the first argument register; ARM64 returns small structs in registers, large ones through x8. A decompiler that guesses the convention wrong produces a plausible but false signature.
- (g) Flavour variants worth knowing: ARM EABI hard-float versus soft-float (argument registers versus a software convention), MIPS `o32/n32/n64`, and `x32`. Each changes the argument registers and therefore every recovered prototype.
- (h) The audit consequence: an architecture the auditor cannot read is an attack surface the auditor cannot price. The same bug in a driver on MIPS (delay slots, unaligned access) is a different bug than on x86-64, which is why the kernel lecture's vendor drivers are the place to apply this section.

Disassemblers

1. Possibly show how disassemblers are written.
 - (a) Ability to decode all instruction encodings of a target is the easy part: table-driven decoders map opcodes to mnemonic/operand tuples. The hard part is control: what bytes are instructions and where functions begin.
 - (b) Linear sweep: decode from the start of the section to the end. Simple, breaks on data-in-code, on padding, on jump tables that contain addresses, and on relocation stubs.
 - (c) Recursive descent: start at known entry points, follow control flow (branches), avoid decoding outside reachable ranges. Misses indirect jumps/calls whose target is computed; hard cases require value-set analysis or speculation.
 - (d) Symbol recovery: from symbol tables when present, from imported function signatures otherwise (PLT resolution), from calling conventions and known library signatures (FLIRT-style signature matching) otherwise.
 - (e) Ambiguity in Thumb/ARM mode switching, x86 variable length instruction overlap, and embedded padding are ordinary disassembler failure modes: treat disassembler output with some skepticism.
 - (f) Symbol recovery, in order of preference: the symbol table when present; the dynamic symbol table when only imports/exports survive; import stubs and their call sites (PLT/GOT, Mach-O stubs, PE IAT); library signatures and pattern databases (FLIRT, Lumina, FunctionID) when nothing else remains; and, last, function-boundary heuristics (prologue patterns, alignment, noreturn calls, unwind tables).
 - (g) The noreturn heuristic is why a call to `abort` or `exit` ends a function without a return instruction; the counterpart is a tail call, which begins with a jump and no prologue. Both are common false function boundaries.
 - (h) Jump tables are the recurring data-in-code case: a `switch` compiles to a table of addresses loaded through a scaled index, and a linear sweep will decode the table as instructions. Recursive descent avoids the table but must know that the indirect jump is a jump.
 - (i) x86 instruction overlap is the linear sweep's second failure mode: a byte sequence can decode two ways depending on the start offset, so a single wrong byte misaligns the whole rest of the section.
 - (j) Disassembler engines are libraries now (Capstone, Zydis, XED); building a custom tool means choosing one and supplying the boundary heuristics, not writing the decoder.

Decompilers

1. Possibly show how decompilations are not unique.
 - (a) A decompiler recovers a program consistent with the binary semantics; many source-level programs are consistent, and there is no canonical reverse mapping.

- (b) Redeclaration (many C programs compile to the same binary), undefined structure layout, and unreachable code are the largest classes of ambiguity.
 - (c) The output is a hypothesis, not truth; when a decompiler and a debugger disagree, the debugger (or a run) is right.
2. Show pitfalls of decompilers, like function inlining and optimizations
- (a) Inlining erases the call edge: two functions appear as one, types are mixed, the decompiler's inferred prototype is wrong if the call disappeared.
 - (b) Switch tables are expanded to chains of compares, arrays fold to loops with unrolled bodies: readable structure is lost.
 - (c) Optimization is a transformation, not a document: branch folding produces single-trace functions that appear to have no condition where the source had many.
 - (d) Heap/stack register usage is a large area of decompiler weakness: fastcall setup, untyped pointer casts, and goto-based state machines all produce wrong-looking output.
 - (e) Trust hierarchy: hand disassembly > static decompiler > (both wrong when the binary is packed, self-modifying, or has just unknown encodings).
 - (f) How a decompiler works, in one line: lift each instruction into a small IR, build the control-flow graph, run SSA construction, then run the classic compiler optimizations backwards (copy propagation, dead-code elimination, expression folding, structure recovery) to print C. Every step is an inference, and each has a known failure mode.
 - (g) Recurring failure modes to expect: `setjmp/longjmp` and exception edges (the CFG loses the second path), `alloca` and variable-length arrays (a dynamic stack frame), inline assembly (opaque to the lifter), tail calls (a missing call edge), `varargs` (a prototype that hides the real arguments), and `__int128`/floating point (operations the IR may not model exactly).
 - (h) Structure and type recovery is the weakest part: a decompiler invents a struct where the source had an array of scalars, or an `int` where the source had a pointer. Treat every recovered type as a hypothesis to be checked against a run.
 - (i) Decompiler output is still the best starting document for a large binary: it is searchable, annotatable, and re-nameable. The discipline is to record the confidence of each recovered name and type rather than to trust the printed C.

Approach: How to Structure a Reversing Session

1. Triage first: `file`, `otool -L/readelf -d`, `strings`, `packer/entropy` check. What is it, for what platform, packed or not?
2. Map the surface: imports, exported functions, entry points, and main loops. Locate input processing (syscalls, file/network reads) since that is where attackable behavior lives.

3. Automate what you can: scripting disassemblers to bucket functions by size, complexity, and xref count focuses human attention (same logic as the source auditing lecture's "where to spend limited time").
4. Dynamic: attach a debugger, provide real input, and observe execution; the interplay of static and dynamic is what makes reversing fast.
5. Document as you go: rename functions, label structures, leave notes in your tool. Reversing is cumulative work; a well-labeled IDB is the deliverable.
6. The triage steps in order, with the tool for each: `file` and the magic bytes; `binwalk` for firmware and container images; an entropy plot and section table for packing; `strings`/`FLOSS` for strings (including obfuscated ones); a YARA scan for known malware and components; and imports plus the entry point for the program's shape.
7. When the static picture stalls (packed, self-modifying, encrypted strings), go dynamic: run it under a debugger or an emulator (`Qiling`/`Unicorn`/`speakeasy`), let it unpack itself, then dump the memory image and reverse the dump. This is the reversing analogue of the fuzzing module's "skip the outer layer" rule.
8. The reversing deliverable is a written understanding, not a screenshot of a decompiler: the call graph, the input parser's structure, the object layouts, and the invariants the code assumes. That document is what the exploit and report sections of this course consume.
9. Reversing is iterative with the other modules: a reversing session produces the entry points for the fuzzing module's harness, the breakpoints and expected invariants for the debugging module, and the mitigation and layout facts for the exploitation module.

Reversing Examples

1. Example x86, x86_64, ARM, ARM64, MIPS and PPC. Compile a non-trivial (but smaller) program and compare architectures; the lab document walks a live session on the course's challenge binary (`binary_re/targets/checks.c`).

- (a) The same C source, compiled at `-O0` for two architectures (clang on macOS: `clang -arch x86_64 -arch arm64 -c hello.c`):

x86_64	ARM64
<code>pushq %rbp</code>	<code>sub sp, sp, #0x20</code>
<code>movq %rsp, %rbp</code>	<code>stp x29, x30, [sp, #0x10]</code>
<code>subq \$0x10, %rsp</code>	<code>add x29, sp, #0x10</code>
<code>leaq "...(%rip), %rdi</code>	<code>adrp x0, l_.str@PAGE; add x0, x0, ...</code>
<code>callq _printf</code>	<code>bl _printf</code>
<code>xorl %eax, %eax</code>	<code>ldr w0, [sp, #0x8]</code>
<code>popq %rbp; retq</code>	<code>ldp x29, x30, [sp, #0x10]; add sp, #0x20; ret</code>

- (b) Read it: x86 stores the frame pointer on the stack at every call (the return address is a stack pop away), ARM64 keeps the return address in x30 (LR) and must save it in the prologue — the `stp/ldp` pair writes/loads both callee-saved registers in one instruction.

- (c) Note the ARM64 pattern `adrp/add`: PC-relative 4K-page addressing needs two instructions to form a 64-bit address; on `x86_64` a single RIP-relative `lea` handles it. Recognizing these idiom pairs is half of reading new architectures.
 - (d) Follow the call convention: `bl` writes the return address into `x30` (LR); `ret` jumps via `x30`. That is why overwriting the saved `x30` slot in a stack buffer overflow redirects execution on ARM64 the same way overwriting the saved return address does on `x86`.
2. I'd like real world examples here.
- (a) Take a known CVE and its patch, and reproduce the bug function from the pre-patch binary; identify the missing bounds check in the disassembly. Good practice targets with published writeups: OpenSSL CVE-2014-0160 (Heartbleed), Stagefright `libstagefright.so` (CVE-2015-3864, integer overflow in the MP4 parser), AppWeb CVE-2018-8715 or older auth-bypass issues.
 - (b) Exercises: find the comparisons guarding the vulnerable path (the bound check that is missing or wrong), the call into `memcpy`-like routines, and the length argument. Diff patched and unpatched versions side by side with a binary diff tool (BinDiff, Diaphora) to see the fix as code motion.
3. Perhaps tear down an embedded system and then show the following reversing.
- (a) Embedded teardown flow: obtain firmware (vendor download, SPI flash dump, UART shell), unpack (`binwalk` for filesystem images: SquashFS, JFFS2, CramFS), identify binaries (`busybox` plus vendor daemons), and audit the vendor-written parts (compare the source auditing lecture's "custom code is where the bugs are").
 - (b) Common bug classes found this way: hardcoded credentials, unauthenticated update acceptance, command injection through configuration values, stack overflows in command parsers.
4. Show example of an application process doing network stuff.
- (a) Reverse triage of a network daemon: find `socket/bind/accept` (or higher wrappers), follow the buffer that `recv` fills into parsing functions, and identify length fields copied through.
 - (b) Using a debugger: break on the parse entry with a malformed client packet; watch parameter registers under the target's calling convention. This is the binary-side analogue of the AppWeb `parseHeaders()` lab in the source auditing material.
5. Show example of reversing the bootloader to setup a chip.
- (a) Boot chains: mask ROM boot (immutable, vendor loaded) verifies and loads the bootloader (U-Boot, LittleKernel, vendor ones) which verifies and loads the kernel — verification done by signature check or hash against e-fuses/OTP areas of the chip.
 - (b) Reversing the bootloader usually means: find the signature verification function, identify what it compares (and whether the comparison is early-exit or constant-time), and establish whether failure paths fall through to by-the-same-code loading. Documented cases (e.g. early Android boot ROM bugs) made this class of bug historically available.

6. Other ideas, reverse a proprietary filesystem, proprietary algorithm?

- (a) Proprietary filesystem: reverse the on-disk format by starting with mount logic and its section parsing; document the superblock and inode layout as C structs in your tool; compare against garbage file-system images you create with partially randomized bytes.
- (b) Proprietary algorithm (custom crypto, license checking, game save obfuscation): find the entry by following its caller context (where data is transformed), then instrument inputs/outputs to get pairs of (plaintext, ciphertext) and infer the transformation; static plus dynamic yields the fastest recoveries.

7. Reverse a fuzzing crash back to source (the pipeline the fuzzing module leaves open): take a minimized input, break on the parser entry, and read the length and pointer arguments in the registers. The reversing session answers the two questions the sanitizer report only states: which check is missing, and what the attacker controls.

- (a) The specific exercise is the course's own target: `lab_target.c` and `checks.c` in this repo are small enough to reverse by hand and layered enough to teach the triage order (magic, length, checksum, then the bug).

8. Reverse a driver's `ioctl/mmap` without source, which is the vendor-driver case from the kernel lecture: find the `file_operations` table through the driver's registration call, then read each verb's argument copy and bounds check. The `model_driver.c` companion in the Linux kernel module is the same shape with source.

9. Diff reversing: when a patch exists, diff the patched and unpatched binaries (BinDiff, Diaphora, radiff2) and read the change as code motion. The added check names the bug, which is the fastest root cause in the whole course.

10. Reversing a protocol implementation from a captured session (the network side of the fuzzing module): locate the `recv` call site, name the message parser, and rebuild the wire format as a grammar. That grammar is the generative fuzzer's input.

Listing 1: ARM64 slice of `hello.c` at `-O0` (`objdump -d`)

```

1  _main:
2      0: ff 83 00 d1 sub sp, sp, #0x20
3      4: fd 7b 01 a9 stp x29, x30, [sp, #0x10]
4      8: fd 43 00 91 add x29, sp, #0x10
5      c: 08 00 80 b9 mov w8, #0x0
6     10: e8 0b 00 b9 str w8, [sp, #0x8]
7     14: bf c3 1f b8 stur wzr, [x29, #-0x4]
8     18: 00 00 00 90 adrp x0, l_.str@PAGE
9     1c: 00 00 00 91 add x0, x0, l_.str@PAGEOFF
10    20: 00 00 00 94 bl _printf
11    24: e0 0b 40 b9 ldr w0, [sp, #0x8]
12    28: fd 7b 41 a9 ldp x29, x30, [sp, #0x10]
13    2c: ff 83 00 91 add sp, sp, #0x20
14    30: c0 03 5f d6 ret

```

Conclusion

Reversing is the complement of source auditing: the same analysis questions (where does input enter, how is it validated, what objects live where) asked without source code. Modern tooling (decompilers, scriptable disassemblers, dynamic instrumentation) has moved the bottleneck from reading assembly to understanding program structure and data flow, which is a skill transferable from source auditing. Fall back on the source auditing methods whenever a decompiler lead dead-ends; fall back on reversing whenever the source lies.

Two habits carry the whole module. First, name the format, the architecture, and the compiler before reading any code, because each fixes what the code can look like and what the mitigations cost. Second, treat every recovered name, type, and prototype as a hypothesis with a confidence, and check the ones the exploit depends on against a run. The reversing deliverable is a written, checkable understanding, not a decompiler window.

References

1. The Art of Software Security Assessment, McDonald, Schuh & Dowd (bugs from the auditor's perspective)
2. Apple's OS X ABI Mach-O File Format Reference (object format ground truth)
3. <https://www.capstone-engine.org> (disassembly engine used by many tools)
4. <https://www.jetbrains.com/help/decompiler> (and Ghidra: <https://ghidra-sre.org>)
5. <https://rada.re> (radare2), <https://rizin.re>
6. The Shellcoder's Handbook, Wiley — platform specific exploitation basics
7. Practical Malware Analysis, Sikorski & Stolz
8. <https://www.fuzzingbook.com> (uses reversing to seed fuzzers)
9. ELF specification (refspecs.linuxfoundation.org), Microsoft PE/COFF specification (docs.microsoft.com/windows/win32/debug/pe-format)
10. DWARF Debugging Information Format (dwarfstd.org) and the Itanium C++ ABI (itanium-cxx-abi.github.io)